

Diseño y Desarrollo de Software

LABORATORIO N° 05

Navegación



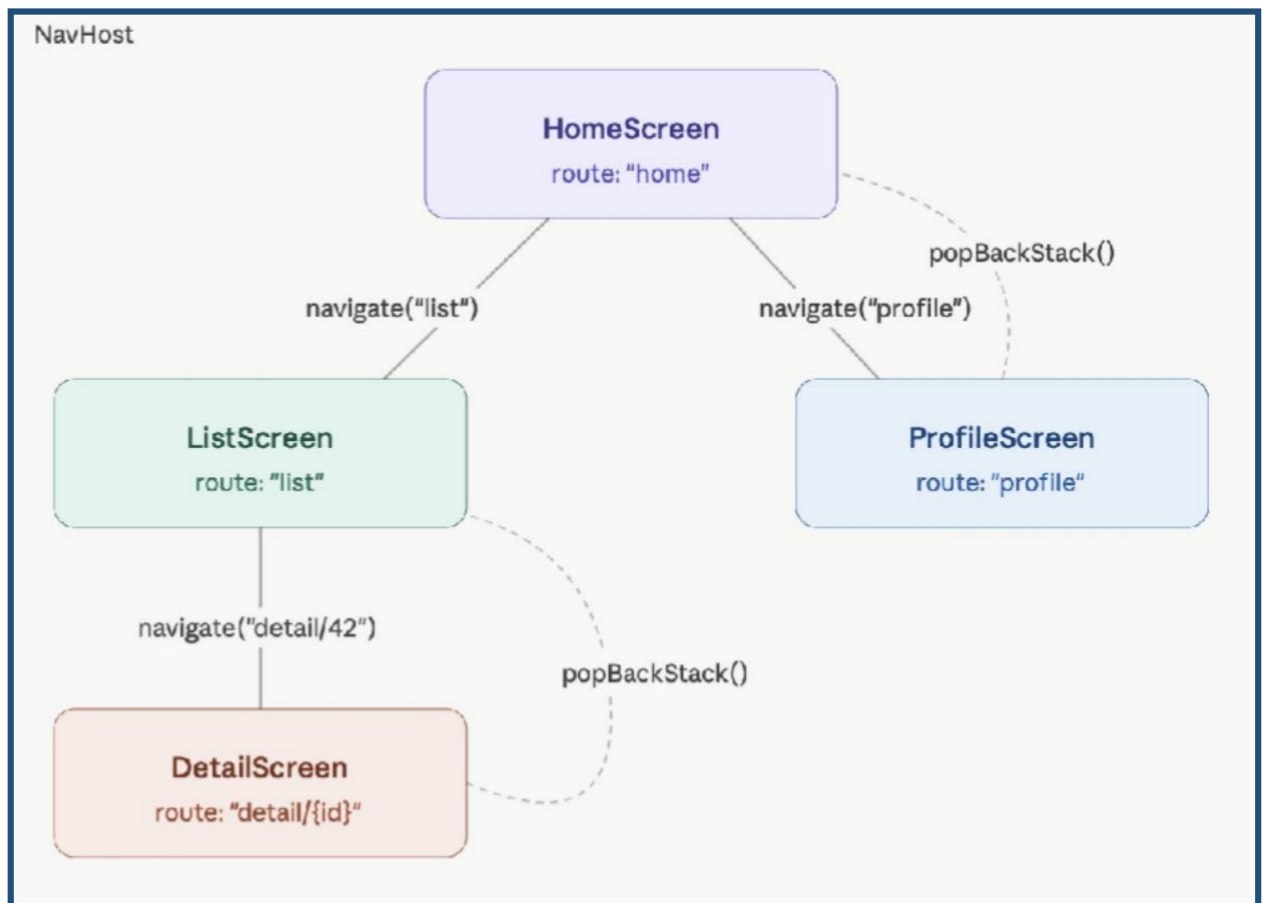
DOCENTE:

JUANLEÓN

CURSO:

**Programación
Móviles**

Actividad : Navegación en JetpackCompose



1. Crear el proyecto en Android Studio

CONFIGURACIÓN AL CREAR

AS New Project → **Empty Activity**

- ☐ Name: NavLab
- ☐ Package: com.tuapp.navlab
- ☐ Language: **Kotlin**
- ☐ Minimum SDK: **API 24**
- ☐ Build configuration: **Kotlin DSL**

ESTRUCTURA INICIAL QUE GENERA ANDROID STUDIO

KT MainActivity.kt — ya existe, la modificaremos

GR build.gradle.kts (módulo :app) — añadir dependencia

Paso	Actividad
1	Crear proyecto
2	Dependencia navigation
3	Estructura de paquetes y archivos
4	Screen.kt (sealed class)
5	AppNavigation.kt + MainActivity
6	HomeScreen + ListScreen
7	DetailScreen + ProfileScreen
8	Ejecutar y verificar flujos

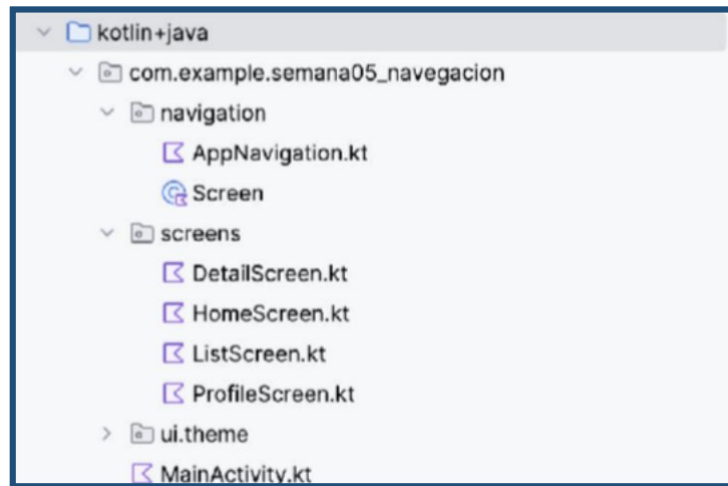
2. Archivo : build.gradle.kts(Module:app)

```
dependencies {
    // ← AÑADIR ESTA LÍNEA (única nueva)
    implementation("androidx.navigation:navigation-compose:2.7.7")

    // Las siguientes ya vienen por defecto
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.compose.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.compose.ui.test.junit4)
    debugImplementation(libs.androidx.compose.ui.tooling)
    debugImplementation(libs.androidx.compose.ui.test.manifest)
}
```

Después de guardar el archivo, hacer clic en Sync Now en el banner amarillo. Sin sync, las clases de navegación no estarán disponibles.

3. Crear la Estructura de paquetes y Archivos



4. **Screen.kt:** rutas con sealed class.

Sealed class es una característica pura de kotlin

```

// Clase sellada que actúa como contrato central de navegación.
// Recibe "route" como parámetro - es el identificador único de cada pantalla.
// Al ser sealed, el compilador conoce todas las rutas posibles en tiempo de compilación.
24 Usages 4 Inheritors
sealed class Screen(val route: String) {

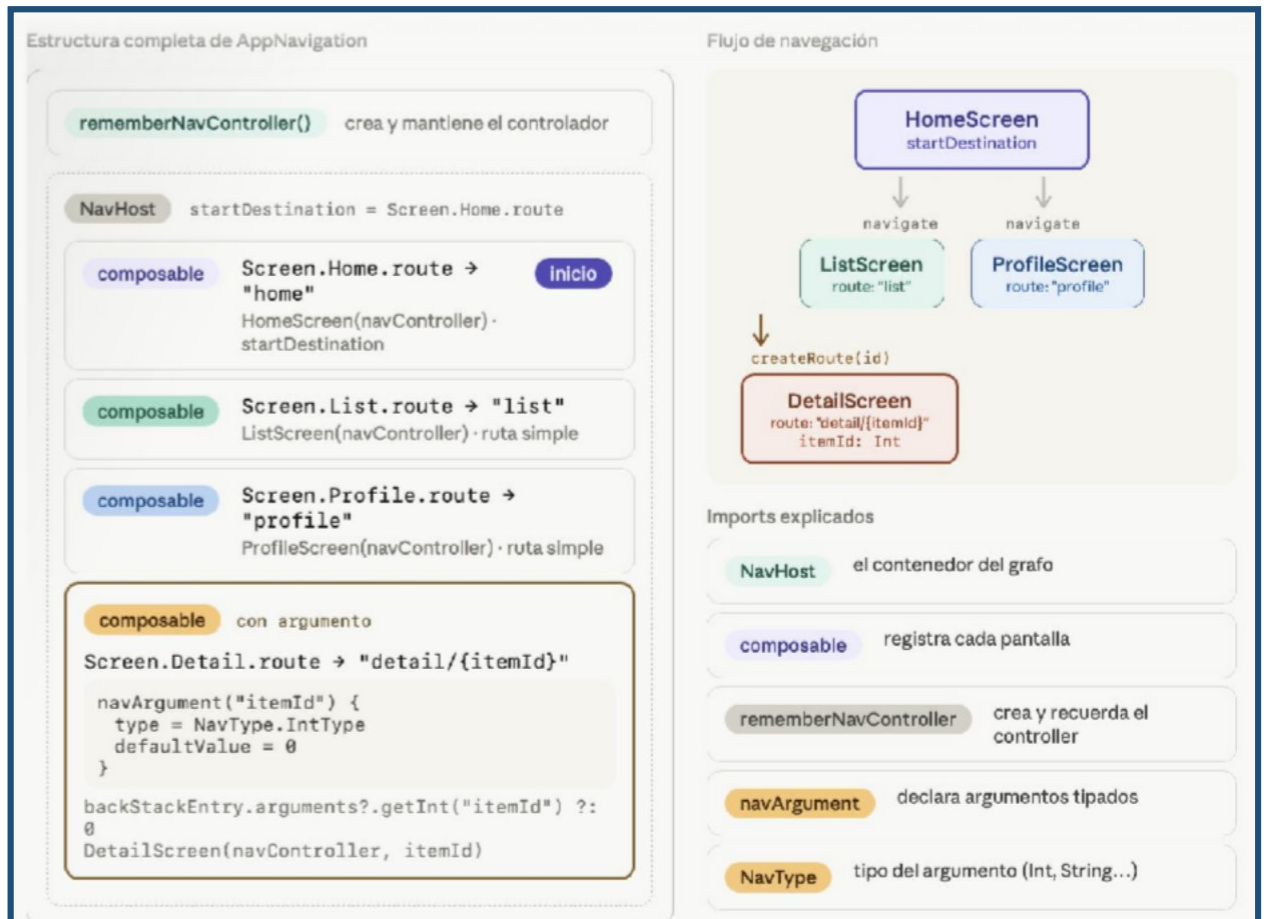
    // Desarrollado por: Juan León
    // Pantalla de inicio - punto de entrada de la app
    6 Usages
    object Home : Screen( route = "home")
    // Pantalla que muestra la lista de elementos
    3 Usages
    object List : Screen( route = "list")
    // Pantalla del perfil del usuario
    3 Usages
    object Profile : Screen( route = "profile")
    // -----
    // RUTA CON ARGUMENTO
    // {itemId} es el placeholder que Navigation reemplaza
    // con el valor real al momento de navegar
    // -----

    3 Usages
    object Detail : Screen( route = "detail/{itemId}") {

        // Construye la ruta final sustituyendo el placeholder por el valor real.
        // Ejemplo: createRoute(5) → devuelve "detail/5"
        // Este String es el que se pasa a navController.navigate(...)
        1 Usage
        fun createRoute(itemId: Int): String = "detail/${itemId}"
    }
}

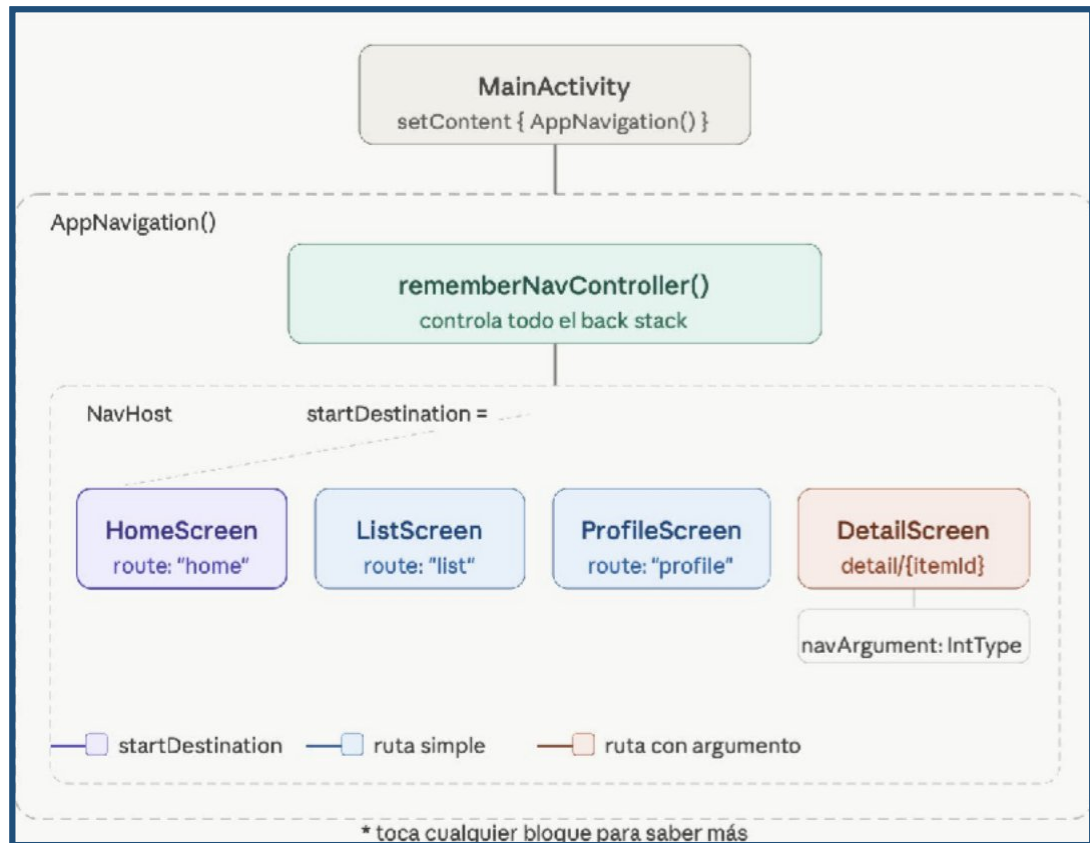
```

5. AppNavigation.kt



6. MainActivity.kt

Este archivo es el **corazón de la navegación**. Todo lo que necesita la app para moverse entre pantallas vive aquí. Se divide en dos partes:



```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.semana05_navegacion.navigation.AppNavigation
import com.example.semana05_navegacion.navigation.Screen
import com.example.semana05_navegacion.screens.DetailScreen
import com.example.semana05_navegacion.screens.HomeScreen
import com.example.semana05_navegacion.screens.ListScreen
import com.example.semana05_navegacion.screens.ProfileScreen
import com.example.semana05_navegacion.ui.theme.Semana05_NavegacionTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppNavigation()
        }
    }
}

```

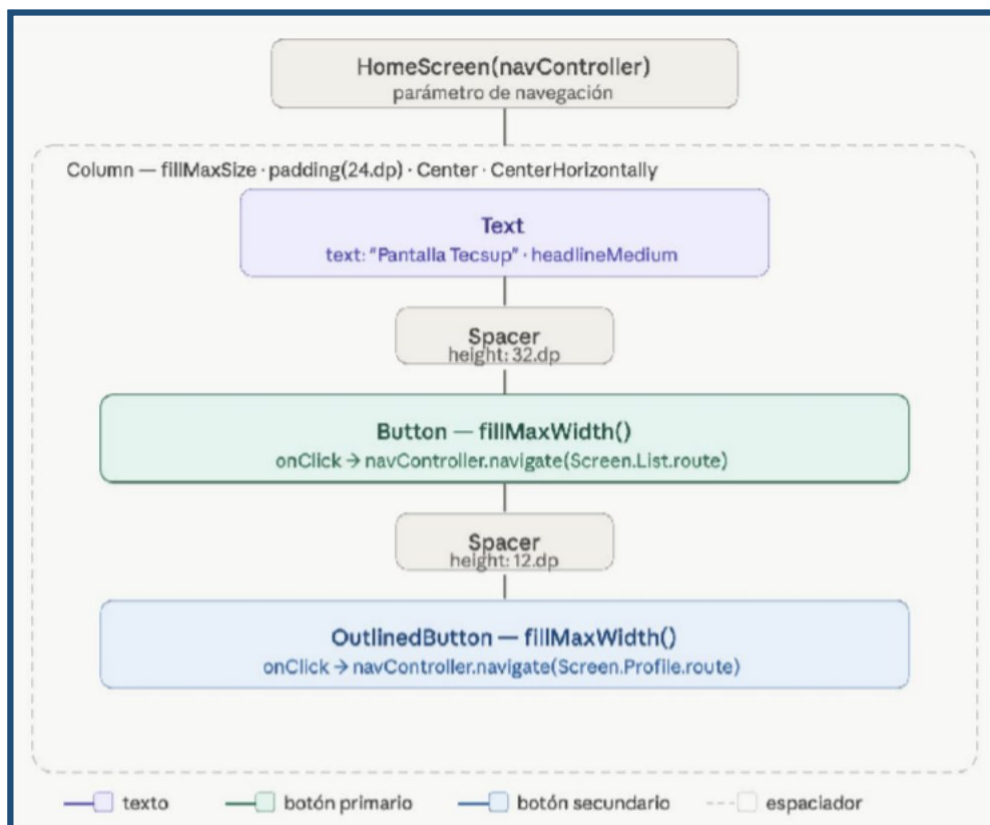
```

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = Screen.Home.route
    ) {
        composable(Screen.Home.route) {
            HomeScreen(navController)
        }
        composable(Screen.List.route) {
            ListScreen(navController)
        }
        composable(Screen.Profile.route) {
            ProfileScreen(navController)
        }
        composable(
            route = Screen.Detail.route,
            arguments = listOf(
                navArgument( name = "itemId" ) {
                    type = NavType.IntType
                    defaultValue = 0
                }
            )
        ) { backStackEntry ->
            val itemId = backStackEntry.arguments?.getInt( key = "itemId" ) ?: 0
            DetailScreen(navController, itemId)
        }
    }
}

```

HomeScreen.kt



Vista en dispositivo

Pantalla Tecsup

Ver lista de elementos

Mi perfil

Qué genera cada línea de código

Text headlineMedium
text = "Pantalla Tecsup"

Spacer espacio vacío vertical
height = 32.dp

Button relleno azul
onClick → navigate(
Screen.List.route)

Spacer espacio vacío vertical
height = 12.dp

OutlinedButton solo borde
onClick → navigate(
Screen.Profile.route)

Column centra todo con
Arrangement.Center vertical
Alignment.CenterHorizontally

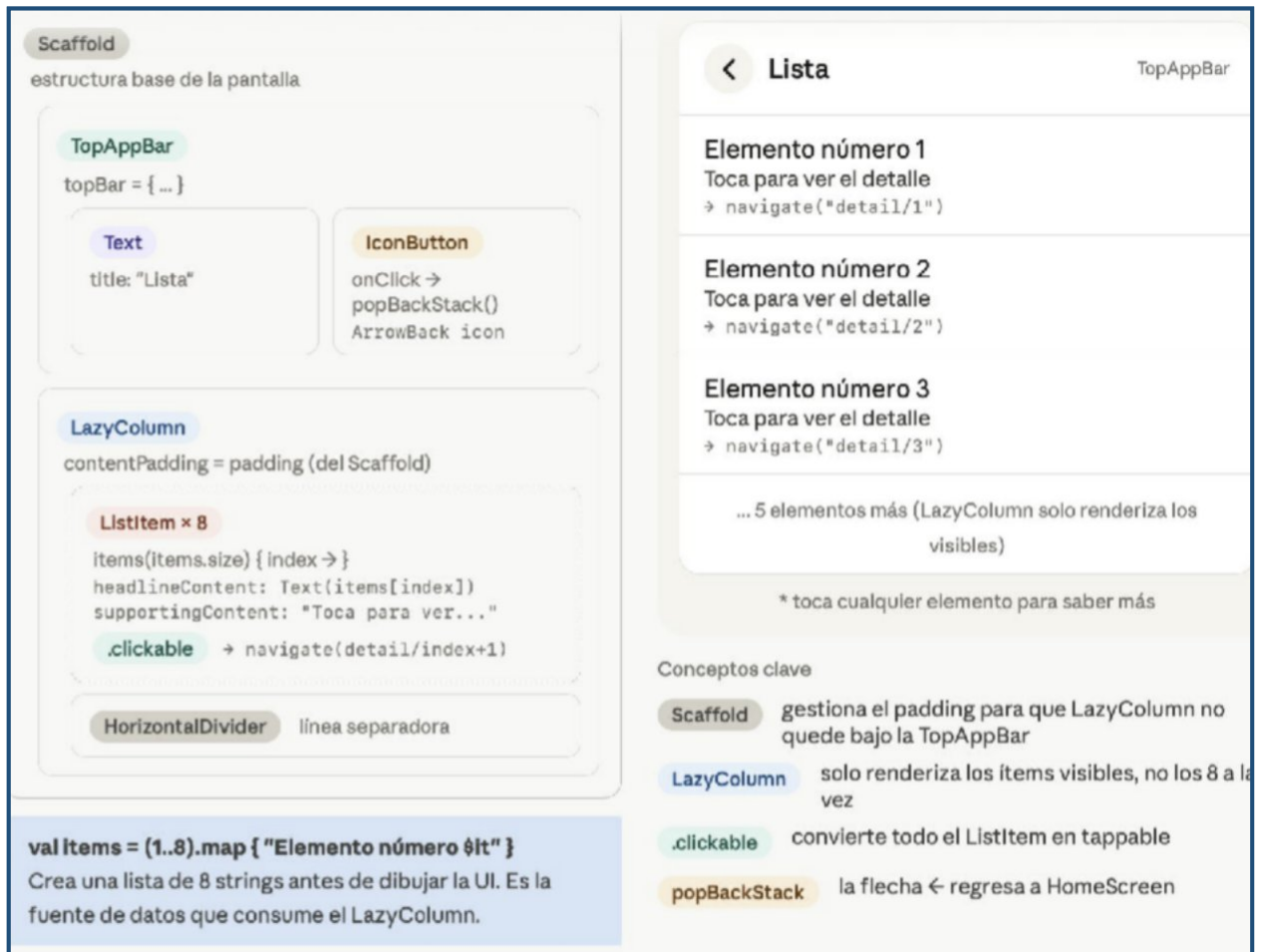
```

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.semana05_navegacion.navigation.Screen

@Composable
fun HomeScreen(navController: NavController) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Pantalla Tecsup",
            style = MaterialTheme.typography.headlineMedium
        )
        Spacer(modifier = Modifier.height(32.dp))
        Button(
            onClick = { navController.navigate(Screen.List.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Ver lista de elementos")
        }
        Spacer(modifier = Modifier.height(12.dp))
        OutlinedButton(
            onClick = { navController.navigate(Screen.Profile.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Mi perfil")
        }
    }
}

```

ListScreen.kt



```
// navigate() - SUMA una pantalla al stack
navController.navigate(Screen.Detail.route)
// stack: Home → List → Detail

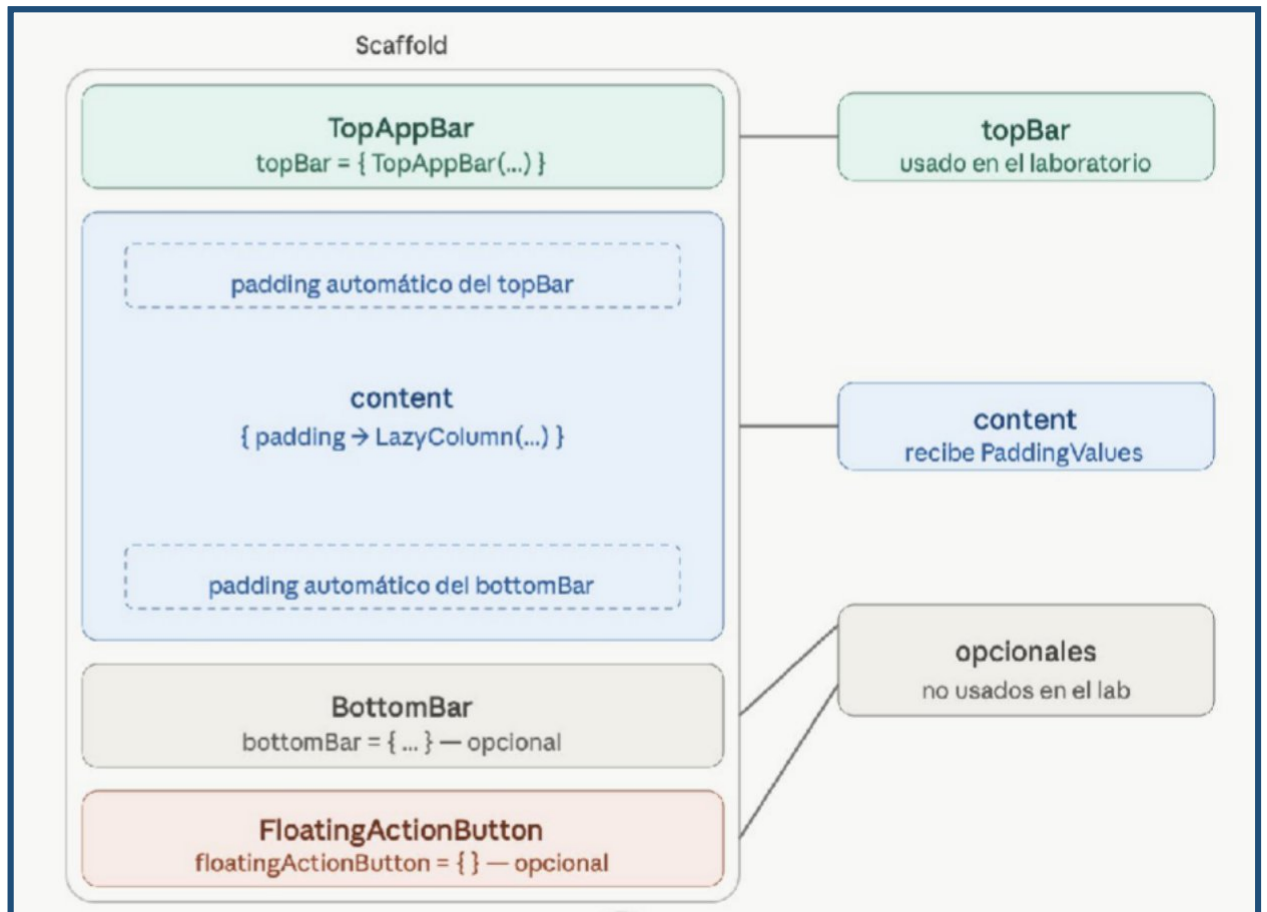
// popBackStack() - RESTA la pantalla del tope
navController.popBackStack()
// stack: Home → List
```

TopAppBar :es el componente de Material3 que dibuja la **barra superior de una pantalla**. Es la barra que ves en casi todas las apps de Android con el título y los botones de acción.

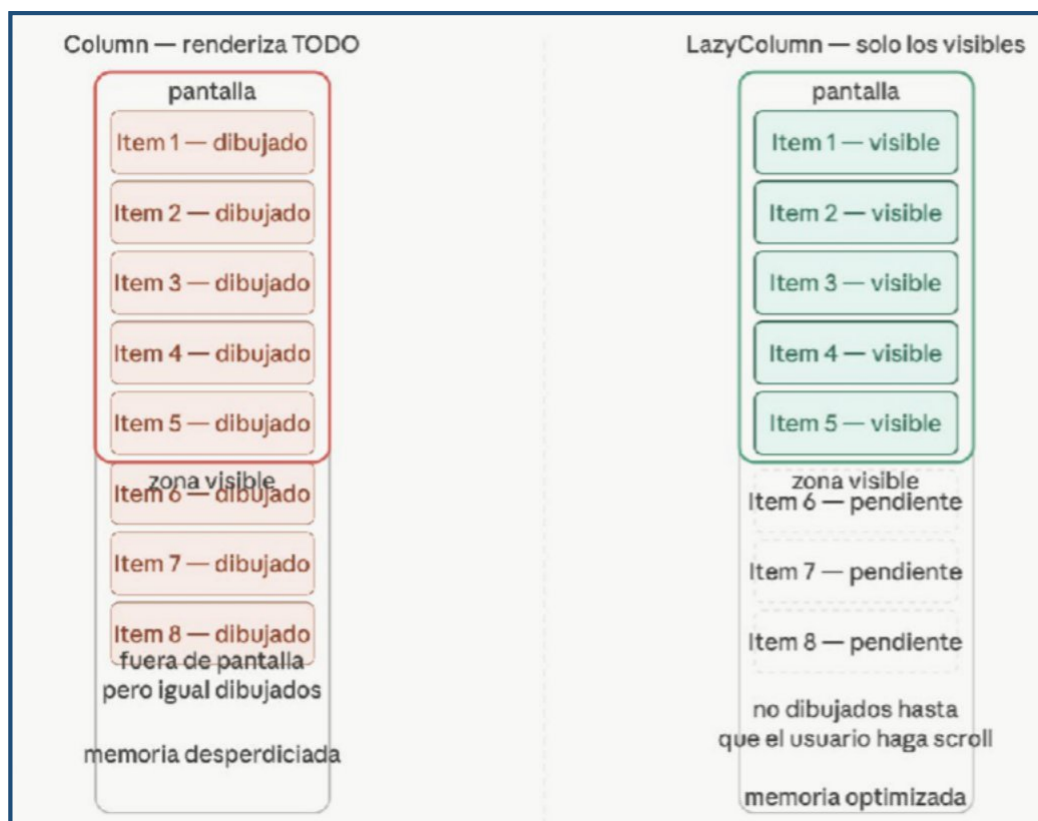
Por qué necesita @OptIn

TopAppBar está marcada como `@ExperimentalMaterial3Api` por Google porque su comportamiento con scroll todavía puede cambiar en versiones futuras.

Scaffold es el **esqueleto de una pantalla completa** en Material3. Se encarga de organizar automáticamente todos los elementos típicos de una pantalla Android sin que tengas que calcular posiciones manualmente.



LazyColumn : solo dibuja los elementos que el usuario puede ver en pantalla en ese momento.



```

import androidx.compose.foundation.clickable
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.ListItem
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.NavController
import com.example.semana05_navegacion.navigation.Screen

```

```

@OptIn(ExperimentalMaterial3Api::class)

```

```

@Composable

```

```

fun ListScreen(navController: NavController) {
    val items = (1..8).map { "Elemento número $it" }

```

```

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Lista") },
                navigationIcon = {
                    IconButton(onClick = { navController.popBackStack() }) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Volver"
                        )
                    }
                }
            )
        }
    )
}

```

```

) { padding ->
    LazyColumn(contentPadding = padding) {
        items(items.size) { index ->
            ListItem(
                headlineContent = { Text(items[index]) },
                supportingContent = { Text("Toca para ver el detalle") },
                modifier = Modifier.clickable {
                    navController.navigate(
                        Screen.Detail.createRoute(index + 1)
                    )
                }
            )
            HorizontalDivider()
        }
    }
}
}
}
}

```

DetailScreen.kt

Árbol de composables

Scaffold estructura base

- TopAppBar
 - Text "Detalle del elemento"
 - IconButton popBackStack()
- Column .padding(padding),padding(24.dp)
 - Text "Elemento #\${itemid}" · headlineSmall
 - Spacer height:12.dp
 - Card fillMaxWidth()
 - Column padding(16.dp)
 - Text "ID recibido: \$itemid" · bodyLarge
 - Spacer height: 8.dp
 - Text "Este valor llegó..." · bodyMedium

itemid: Int es el argumento que viaja desde ListScreen. No es un String — llega directamente como Int porque así se declaró con NavType.IntType en el NavHost.

Vista en dispositivo — Itemid = 3

< Detalle del elemento TopAppBar

Elemento #3

ID recibido: 3

Este valor llegó como argumento tipado Int desde el NavHost.

Card — fillMaxWidth()

* toca el botón < y la Card para saber más

Lo nuevo respecto a ListScreen

- Card contenedor elevado con fondo propio — agrupa información relacionada visualmente
- Itemid: Int argumento recibido del NavHost ya convertido a Int, listo para usar en la UI
- doble padding .padding(padding) respeta el Scaffold y .padding(24.dp) añade margen interior

Card es un contenedor con fondo elevado propio. Visualmente agrupa la información del ítem seleccionado separándola del fondo de la pantalla.

itemid: Int es el argumento que llegó tipado desde el NavHost. No es un String que hay que convertir, llega directamente como Int porque así se declaró con NavType.IntType

```

import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.semana05_navegacion.navigation.Screen

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailScreen(navController: NavController, itemId: Int) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Detalle del elemento") },
                navigationIcon = {
                    IconButton(onClick = { navController.popBackStack() }) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Volver"
                        )
                    }
                }
            )
        }
    ) {
        ) { padding ->
            Column(
                modifier = Modifier
                    .padding(padding)
                    .padding(24.dp)
            ) {
                Text(
                    text = "Elemento #${itemId}",
                    style = MaterialTheme.typography.headlineSmall
                )
                Spacer(modifier = Modifier.height(12.dp))
                Card(modifier = Modifier.fillMaxWidth()) {
                    Column(modifier = Modifier.padding(16.dp)) {
                        Text(
                            text = "ID recibido: $itemId",
                            style = MaterialTheme.typography.bodyLarge
                        )
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = "Este valor llegó como argumento tipado Int " +
                                "desde el NavHost.",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            }
        }
    }
}

```

ProfileScreen.kt

```

Column fillMaxSize - padding(24.dp)
Arrangement.Center - Alignment.CenterHorizontally

Text "Mi Perfil" - headlineMedium

Spacer height: 8.dp

Text "Juan León Suiyon" - bodyLarge
color: onSurfaceVariant (gris)

Spacer height: 32.dp

Button fillMaxWidth()
navigate(Screen.Home.route) {
    popUpTo(Home) {
        inclusive = true
    }
}
        
```

Lo más importante de esta pantalla: el `popUpTo` dentro del `navigate()`. Sin él el stack se llenaría de Homes apilados cada vez que el usuario va al inicio.

Esta pantalla es la única que **no tiene TopAppBar**. No necesita botón de volver porque el botón del dispositivo funciona igual y el Button del inicio reemplaza ese rol.

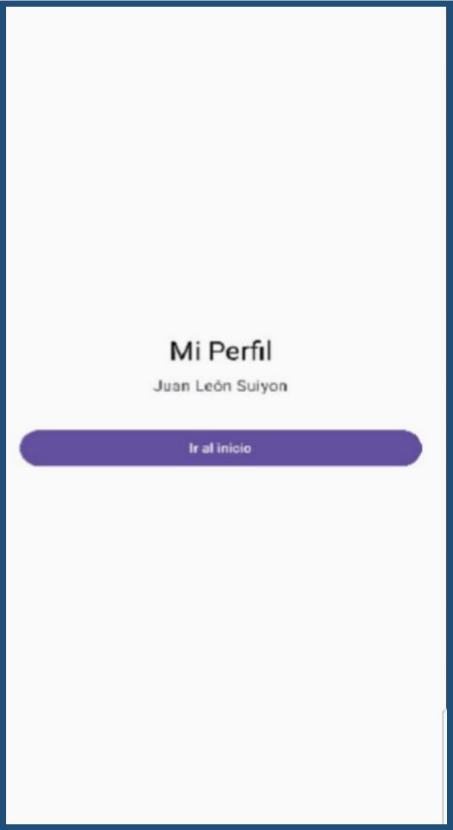
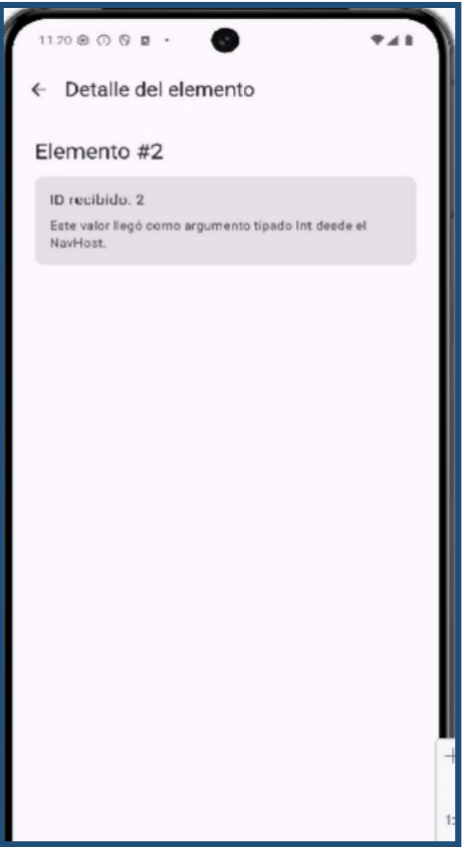
```

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.semana05_navegacion.navigation.Screen
    
```

```

@Composable
fun ProfileScreen(navController: NavController) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Mi Perfil",
            style = MaterialTheme.typography.headlineMedium
        )
        Spacer(modifier = Modifier.height(8.dp))
        Text(
            text = "Juan León Suiyon",
            style = MaterialTheme.typography.bodyLarge,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(modifier = Modifier.height(32.dp))
        Button(
            onClick = {
                navController.navigate(Screen.Home.route) {
                    // Limpia el back stack — evita apilar Homes
                    popUpTo(Screen.Home.route) { inclusive = true }
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Ir al inicio")
        }
    }
}
    
```

Resultado Final:



Utilizando Gemini, realizar el prompt para mejorar la presentacion de la primera parte.

